

Department of Electronics and Telecommunication Engineering

SKILL COMPETENCY EXAMINATION REPORT

Subject: Embedded Systems and RTOS

Title of Project:

**RTOS-Based Smart Health Monitoring System with Fall Detection and
Emergency Alerts**

Name	Himani Deore
Roll Number	312044
PRN	22311497
Class	TY B ENTC
Department	Electronics and Telecommunication Engineering
Subject	Embedded Systems and RTOS
Guide / Examiner	Dr. Shailesh Kulkarni
Academic Year	2025 - 26

1. Abstract

This project presents the design and implementation of a real-time health monitoring system using FreeRTOS on the STM32F401 microcontroller. The system measures vital parameters such as heart rate (BPM) and oxygen saturation (SpO₂) using the MAX30100 sensor and detects fall conditions using the MPU6050 accelerometer.

A touch sensor is implemented using a hardware interrupt mechanism to control the system efficiently. The system continuously displays data on an OLED and generates emergency alerts when abnormal conditions such as fall detection or absence of a valid finger signal are detected.

The system demonstrates real-time multitasking, interrupt handling, synchronization using mutex and semaphore, and priority-based execution.

2. Introduction

In modern healthcare systems, continuous monitoring of vital parameters is essential, especially for elderly and patients with medical conditions. Traditional embedded systems that rely on a single loop are not efficient in handling multiple tasks such as sensing, processing, and display simultaneously.

This project overcomes these limitations by using a Real-Time Operating System (RTOS). The system is divided into multiple tasks that run independently and concurrently, improving responsiveness and system efficiency.

The system behaves like a smart wearable health monitoring device capable of:

- Measuring heart rate and SpO₂
- Detecting falls
- Displaying real-time data
- Generating emergency alerts
- Allowing user control through touch

3. Objectives

The main objectives of this project are:

- To implement real-time health monitoring using RTOS
- To measure heart rate and SpO₂ accurately
- To detect fall conditions using accelerometer data
- To implement interrupt-based user control
- To display real-time data and alerts on an OLED
- To demonstrate multitasking and synchronization

4. Hardware Description

4.1 Microcontroller

The system uses the STM32F401 microcontroller which provides:

- ARM Cortex-M4 core
- High processing speed
- Multiple I2C communication interfaces
- Efficient interrupt handling

4.2 Components Used

- MAX30100 Sensor
Used for measuring heart rate and SpO₂ using optical sensing
- MPU6050 Sensor
Used for detecting motion and fall using acceleration data
- OLED (16x2 with I2C)
Used to display system output
- Touch Sensor
Used as a user input device
- LED (Optional)
Used for indication or alerts

4.3 Communication Interfaces

- I2C1 → MPU6050 and OLED
- I2C3 → MAX30100
- GPIO → Touch sensor and LED

5. Software Design using FreeRTOS

FreeRTOS is used to manage multiple tasks efficiently. Instead of running everything in a single loop, the system is divided into independent tasks that run concurrently. In FreeRTOS, a task is a small independent function that runs concurrently with other tasks. Each task is assigned a priority level, which determines its execution order. The scheduler is responsible for switching between tasks based on priority and timing requirements. This ensures that critical tasks, such as emergency detection, are executed immediately without delay.

5.1 Task-Based Architecture

The system includes the following tasks:

- Touch Task (High Priority)
Handles user input using interrupt and semaphore
Toggles system ON/OFF

- MAX Task (Normal Priority)
Reads sensor data from MAX30100
Calculates heart rate and SpO₂
- MPU Task (Above Normal Priority)
Reads accelerometer data
Detects fall conditions
- Display Task (Default Task)
Updates OLED display based on system state

5.2 Synchronization Mechanisms

- Mutex
Used to protect I2C communication
Ensures only one task accesses the bus at a time
- Semaphore
Used for communication between interrupt and touch task

5.3 Interrupt-Based Touch Control

The touch sensor is implemented using a hardware interrupt (EXTI).

When the touch sensor is activated:

- An external interrupt is triggered
- The interrupt service routine releases a semaphore
- The Touch Task is activated
- The system state is toggled

Flow:

Touch Sensor → Interrupt → ISR → Semaphore → Touch Task → System Toggle

This method ensures:

- Fast response
- Efficient CPU usage
- No continuous polling

6. Working of the System

When the system is turned ON using the touch sensor, the RTOS scheduler activates all tasks. The MAX task continuously reads sensor data and calculates heart rate and SpO₂. Simultaneously, the MPU task monitors acceleration values to detect fall conditions. The display task updates the OLED with the latest information, while continuously checking for

emergency conditions. If any abnormal condition is detected, the display is immediately updated to show an alert.

6.1 System States

The system operates in three main states:

1. OFF State
 - Only touch sensor is active
 - OLED displays:
System OFF
Touch to Start
2. Normal State
 - Sensors are active
 - Displays:
Heart Rate (BPM)
SpO₂ (%)
Signal strength
3. Emergency State
 - Overrides normal display

6.2 Heart Rate and SpO₂ Measurement

- MAX30100 provides IR and Red light signals
- Heart rate is calculated using peak detection
- Time difference between peaks is used to calculate BPM
- SpO₂ is calculated using ratio of Red and IR signals
- Filtering and averaging are applied for stable readings

6.3 Fall Detection

Fall detection is implemented using acceleration data from MPU6050.

Steps involved:

1. Detect free fall (low acceleration)
2. Detect impact (high acceleration)
3. Check no movement for a fixed time
4. Trigger fall alert

This reduces false detection and improves reliability.

6.4 Touch Control

- Touch sensor acts like a power button
- Uses interrupt-based detection
- Toggles system ON/OFF

7. Display System

The OLED displays output based on priority:

Fall Detection (Highest Priority):

FALL ALERT

HELP NEEDED

Emergency Condition (No Finger):

EMERGENCY

Vitals Dropping

Normal Display

HR: XXX SpO₂: XXX

Signal: XXXX

System OFF:

System OFF

Touch to Start

8. Algorithm

1. Initialize system and peripherals
2. Initialize FreeRTOS
3. Create tasks
4. Start scheduler
5. Wait for touch input
6. If system is ON:
 - Read MAX30100
 - Calculate BPM and SpO₂
 - Read MPU6050
7. Detect fall condition
8. Update OLED
9. Check emergency conditions
10. Repeat

9. Results

During testing, the system successfully displayed real-time heart rate and SpO₂ values with stable readings. The touch sensor responded instantly due to interrupt-based implementation. When the finger was removed, the system correctly identified the absence of signal and displayed an emergency message. Fall detection was also successfully triggered during sudden movement, demonstrating the effectiveness of the algorithm.

The system was tested under various conditions:

Normal Operation

- Heart rate and SpO₂ values were displayed correctly
- OLED updated in real time

Touch Control

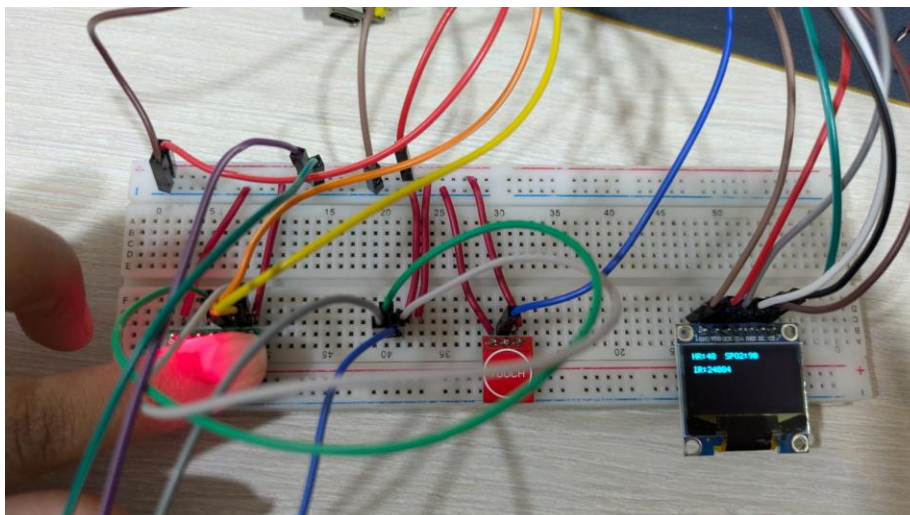
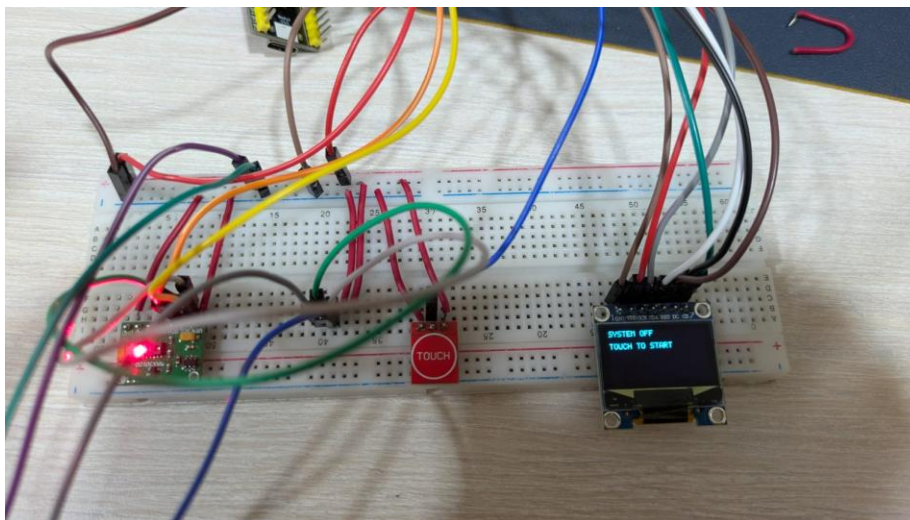
- System toggled ON/OFF instantly
- Interrupt-based response was fast and reliable

No Finger Condition

- When finger was removed:
 - Emergency message was displayed
 - Sensor values reset

Fall Detection

- Sudden movement triggered fall detection
- Emergency alert was displayed correctly



10. Conclusion

This project successfully demonstrates the implementation of a real-time health monitoring system using RTOS. By dividing the system into multiple tasks, efficient multitasking and real-time performance are achieved.

The integration of RTOS, sensor processing, and interrupt-based control makes the system efficient and reliable. This project demonstrates how modern embedded systems can be designed to handle multiple real-time operations effectively. The use of interrupts, mutex, and semaphores ensures proper synchronization and fast response. Compared to traditional systems, this approach provides better scalability, responsiveness, and structured design.

11. Future Scope

- Integration with GSM module for alerts
- Mobile app connectivity (Bluetooth/WiFi)
- Data logging and cloud storage
- AI-based health analysis
- Power optimization for wearable applications